

M295 Backend für Applikationen realisieren

Name des Projekts:

Einkaufsliste

Datum (TT.MM.JJJJ):

24.04.2026

Vorname und Nachname des Autors:

Nico Seiler

Kursnummer:

21-295-F

1. Projektübersicht

In diesem ÜK-Projekt möchte ich eine einfache Einkaufslisten-App entwickeln. Benutzer können sich registrieren, einloggen und ausloggen. Nach dem Login können sie mehrere Einkaufslisten erstellen und verwalten, sowie Artikel innerhalb dieser Listen erfassen, bearbeiten und als erledigt markieren.

Die App besteht aus einem Java-Backend mit REST-Schnittstellen und einer Datenbank, die alle Daten speichern.

2. Projektziel

Ziel ist es ein funktionierendes Backend zu bauen, das folgende Funktionen hat:

- Benutzer können sich registrieren, einloggen und ausloggen.
- Verwaltung von mehreren Einkaufslisten (erstellen, anzeigen, löschen)
- Verwaltung von Artikeln innerhalb einer Liste (hinzufügen, bearbeiten, löschen)
- Markierung von Artikeln als 'erledigt' oder 'offen'

3. Technische Umsetzung

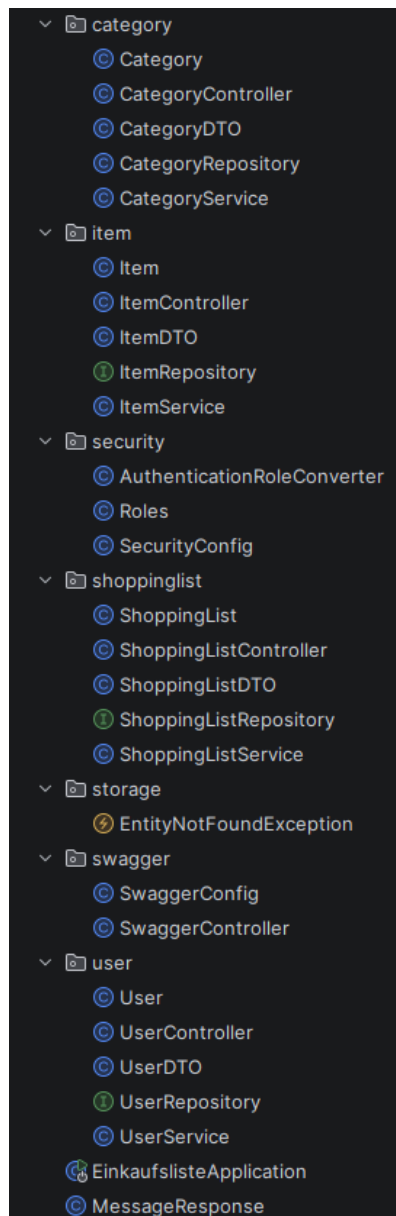
Technologien

Framework	Spring Boot
Sprache	Java
Datenbank	PostgreSQL mit JPA / Hibernate
Authentifizierung	Keycloak
API-Dokumentation	Swagger UI
Build-Tool	Maven

Projektstruktur

Der Code ist nach Single-Responsibility-Prinzip in folgende Packages gegliedert:

- item/ - Entity, Repository, Service, Controller für die Artikel
- category/ - Entity, Repository, Service, Controller für die Kategorien
- shoppinglist/ - Entity, Repository, Service, Controller für die Einkaufslisten
- security/ - AuthenticationRoleConverter, Roles, SecurityConfig
- storage/ - EntityNotFoundException
- swagger/ - SwaggerConfig, SwaggerController
- root/ - EinkaufslisteApplication, MessageResponse
- user/ - Entity, DTO, Repository, Service, Controller für die Benutzer



Entitäten

Die Applikation enthält folgende Entitäten:

- Category – id, name, description, color
- Item – id, name, note, quantity, done, category
- ShoppingList – id, name, description, items, owner
- User – id, keycloakId, username, email, firstName, lastName, shoppingLists

REST-Endpunkte

Category:

Methode	Endpunkt	Rolle	Beschreibung
GET	api/category	read	Alle Kategorien abrufen
GET	api/category/{id}	read	Kategorien per ID abrufen
POST	api/category	admin	Neue Kategorie erstellen
PUT	api/category/{id}	update	Kategorie aktualisieren
DELETE	api/category/{id}	admin	Kategorie löschen

ShoppingList:

Methode	Endpunkt	Rolle	Beschreibung
GET	api/shoppinglist	read	Alle Listen abrufen
GET	api/shoppinglist/{id}	read	Liste per ID abrufen
POST	api/shoppinglist	admin	Neue Liste erstellen
PUT	api/shoppinglist/{id}	update	Liste aktualisieren
DELETE	api/shoppinglist/{id}	admin	Liste löschen

User:

Methode	Endpunkt	Rolle	Beschreibung
GET	api/user	admin	Alle Benutzer abrufen
GET	api/user/{id}	read	Benutzer per ID abrufen
GET	api/user/me	read	Eigenes Profil abrufen
PUT	api/user/{id}	admin	Benutzer aktualisieren
DELETE	api/user/{id}	admin	Benutzer löschen

Item:

Methode	Endpunkt	Rolle	Beschreibung
GET	api/shoppinglist/{listId}/item	read	Alle Artikel einer Liste abrufen
GET	api/shoppinglist/{listId}/item/{itemId}	read	Artikel per ID abrufen
POST	api/shoppinglist/{listId}/item	admin	Neuen Artikel zur Liste hinzufügen
PUT	api/shoppinglist/{listId}/item/{itemId}	update	Artikel aktualisieren
DELETE	api/shoppinglist/{listId}/item/{itemId}	admin	Artikel löschen
PATCH	api/shoppinglist/{listId}/item/{itemId}/toggle	update	Artikel als erledigt/offen markieren

Sicherheit

Alle REST-Endpunkte sind über OAuth2 mit JWT-Tokens abgesichert. Die Authentifizierung erfolgt via Keycloak. Es werden drei Rollen verwendet:

- read - Lesezugriff auf alle Ressourcen (GET)
- update - Schreibzugriff zum Bearbeiten (PUT/PATCH)
- admin - Vollzugriff inkl. Erstellen und Löschen (POST, DELETE)

Tests

Für das Projekt wurden JUnit-Tests erstellt:

- **RestControllerTests** – Testet den CategoryController mit Mockito. Alle CRUD-Methoden (getAll, getByld, create, update, delete) werden überprüft.
- **RepositoryTests** – Testet das CategoryRepository über den CategoryService mit Mockito. Alle CRUD-Operationen (insert, find, findAll, update, delete) werden verifiziert.

UML Diagramm

